A DBMS Mini-Project Report on

WILDBOOK

WILDLIFE SANCTUARY & SAFARI RESERVATION MANAGEMENT SYSTEM



Submitted by

Shravya Ganesh Hegde (U25UV24T029103)

V SEM, BTech (CSE)

Under the guidance of

Prof. Pallavi P

Guest Faculty

Department of CSE

UVCE

UNIVERSITY OF VISVESVARAYA COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

K.R. Circle, Bengaluru – 560001

UNIVERSITY OF VISVESVARAYA COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
K.R. Circle, Bengaluru – 560001

CERTIFICATE

This is to certify that **Shravya Ganesh Hegde** of V Semester, B. Tech, Computer Science & Engineering, bearing the register number **U25UV24T029103** has submitted the DBMS Mini-Project Report on **"WILDBOOK – WILDLIFE SANCTUARY & SAFARI RESERVATION MANAGEMENT SYSTEM"**, in partial fulfilment for the DBMS Lab, prescribed by the University of Visvesvaraya College of Engineering for the academic year 2026-27.

Prof. Pallavi P
Guest Faculty,
Dept. of CSE,
UVCE

Dr. Manjula S H
Professor & Chairperson
Dept. of CSE,
UVCE

Examiners:

1. …… 2. ……

ACKNOWLEDGEMENT

I take this opportunity to thank my institution, University of Visvesvaraya College of Engineering, for giving me an opportunity to carry out this project.

I would like to thank Prof. Subhasish Tripathy, Director, UVCE, for providing all the facilities to work on this project. I am indebted to him for being my pillar of strength and inspiration.

I wish to place my grateful thanks to Dr. Manjula S H, Professor and Chairperson, Department of Computer Science and Engineering, UVCE, who helped me to make this project a great success.

It gives me great pleasure to express my gratitude to Prof. Pallavi P, Guest Faculty, Department of Computer Science and Engineering, UVCE, for her valuable guidance and supervision in the course of this project.

I express my sincere thanks to all teaching and non-teaching staff, Department of Computer Science and Engineering, UVCE, for all the facilities that they have provided for successfully completing this project.

I also thank my parents and friends for their continuous support and encouragement.

Shravya Ganesh Hegde
(U25UV24T029103)

ABSTRACT

WildBook is a web-based Wildlife Sanctuary & Safari Reservation Management System designed to simplify how visitors discover sanctuaries, choose safari activities and reserve seats, and how sanctuary authorities manage guides, vehicles and time slots. It automates user registration, activity and slot management, guide and vehicle allocation, booking with attendee details, payment recording and cancellation. By replacing paper registers and phone-based reservation, the system reduces paperwork, avoids overbooking and gives faster and more reliable access to booking information.

The database is designed in MySQL (InnoDB) by converting the ER diagram into tables with the seven conversion rules and then auditing them against 1NF, 2NF, 3NF and BCNF. The design enforces key rules directly in the database: no guide or vehicle can be assigned to two slots at the same time, a slot cannot carry more people than its vehicle can hold, every booking has one payment, and every attendee is recorded individually. Transactions with row locking keep concurrent bookings safe. The platform stores all information securely and allows it to be retrieved quickly whenever needed.

TABLE OF CONTENTS

Title **Page No.**

1. Introduction..... 1
1.1 Introduction..... 1
1.2 Objective..... 1
1.3 Functionality..... 2
1.4 Database Management System..... 2
1.5 MySQL..... 3

2. Literature Review..... 6
2.1 Survey of Existing Systems..... 6
2.2 Proposed System..... 6
2.3 Software Requirements..... 6
2.4 Frontend Technologies..... 7
2.5 Backend Technologies..... 7
2.6 Entity-Relationship (ER) Model..... 8
2.7 Relational Schema Design..... 9
2.8 Normalization..... 11
2.9 API Development..... 14
2.10 Advantages of the Proposed System..... 14

3. Proposed Work..... 15
3.1 Entity Relationship (ER) Model..... 15
3.2 Relational Model..... 16

4. Results..... 18
4.1 Outcomes..... 18
4.2 Screenshots..... 18

5. Conclusion..... 20

6. Bibliography..... 21

CHAPTER 1
INTRODUCTION

1.1 Introduction

WildBook is a Wildlife Sanctuary & Safari Reservation Management System built on a relational database. It is designed to help visitors discover wildlife sanctuaries, choose safari activities and reserve seats for a particular date and time, while giving sanctuary authorities one place to manage activities, guides, vehicles and time slots. The system keeps structured details such as sanctuary information, activity prices, guide and vehicle records, slot schedules, bookings, attendee details and payments in a MySQL database.

The system replaces paper registers, phone calls and walk-in queues with an online reservation process. A visitor can browse sanctuaries and their activities, see the slots that are open, book seats for a group and record the details of every attendee. Behind the scenes the database makes sure that a guide or a vehicle is never allotted to two slots at the same time, that a slot never carries more people than its vehicle can hold, and that a booking never exceeds the seats that are left.

WildBook is developed as a web-based application using React for the frontend, Node.js with the Express framework for the backend, and MySQL (InnoDB) as the database. The system supports three distinct user roles: Visitors, Sanctuary Administrators (Operators), and Site Owner (Forestry Authority), each enforced with role-based access control (RBAC). Development and database management are carried out using Visual Studio Code, MySQL Workbench and Git for version control.

1.2 Objective

- To design and develop an easy-to-use web-based platform that allows visitors to explore wildlife sanctuaries and conveniently reserve safari and other activity slots online.
- To automate record-keeping, slot scheduling and booking, thereby reducing manual work and minimizing errors such as overbooking and double allocation of guides and vehicles.
- To provide a centralized database for managing sanctuaries, activities, guides, vehicles, slots, bookings, and payments efficiently across a streamlined 9-table schema.
- To design the database through proper ER modelling and normalization (up to 3NF/BCNF), so that redundancy and update, insertion and deletion anomalies are avoided.
- To enforce data integrity and security through keys, constraints, triggers, transactions and role-based access.

User Roles in the System

Admin

- Add, update or remove sanctuaries, activities, guides and vehicles.
- Create safari slots by assigning an activity, a guide and a vehicle for a date and time.
- Monitor bookings, attendee lists and payments.
- Maintain database records and ensure smooth operation of the system.

Visitor (User)

- Create and manage a personal account.
- Browse sanctuaries and activities and view the slots that are available.
- Book seats for a slot and enter the details of each attendee.
- Make payment, view booking history and cancel a booking.

Views Available in the System

The user interface provides the following features and functionalities:

- **Welcome Page** – Introduces the WildBook portal and provides the entry point to login and registration.
- **Browse Page** – Lists sanctuaries, their activities and the open slots.
- **Booking Page** – Lets a visitor choose a slot, enter attendee details and complete payment.
- **Admin Dashboard** – Manages sanctuaries, activities, guides, vehicles, slots, bookings and payments.

1.3 Functionality

WildBook aims to make safari reservation simple for visitors and manageable for administrators. The system provides the following core functional modules:

Sanctuary and Activity Management

- Allows the admin to add, update or remove sanctuaries and the activities offered in each of them.
- Displays the description, location and price of every activity to visitors.

Guide and Vehicle Management

- Maintains guide details such as contact number and years of experience.
- Maintains vehicle details such as type and seating capacity, linked to a sanctuary.

Slot Management

- Lets the admin schedule a slot by assigning an activity, a guide, a vehicle, a date and a time.
- Prevents the same guide or vehicle from being allotted to two slots at the same date and time.
- Ensures that the slot capacity does not exceed the capacity of its vehicle.

Booking Management

- Enables visitors to reserve available seats in scheduled safari slots with an atomic attendee headcount and an automated 10-minute seat hold.
- Checks the number of seats still free before confirming a booking.
- Supports flexible cancellation through a dedicated stored procedure (40% fee / 60% refund split) as well as a direct Cancel & Erase booking feature (DELETE /api/bookings/:id) that instantly frees up slot seats.

Payment Management

- Records exactly one payment for each booking, with its amount and status (paid, failed or refunded).
- Maintains a transaction history for transparency and record-keeping.

Authentication & Security

- Provides secure login with password hashing and role-based access for visitors and admins.
- Provides secure token-based authentication (JWT) with bcrypt-hashed credentials and sanitizes sensitive financial transactions.

1.4 Database Management System

A Database Management System (DBMS) is essential for managing large amounts of data efficiently. Our project uses MySQL to ensure data consistency, security and easy retrieval of records related to sanctuaries, slots and bookings.

Why Use a DBMS?

- **Eliminates data redundancy** – A sanctuary name or an activity price is stored in exactly one place.
- **Ensures data integrity** – Prevents inconsistencies such as a booking for a slot that does not exist.
- **Provides structured queries** – Allows fast retrieval of slots, bookings and payment records.

1.4.1 Characteristics of DBMS

- **Self-describing nature** – Metadata about the tables and columns is stored within the database.
- **Data redundancy control** – Duplicate records are avoided by design.
- **Enforces data integrity rules** – Ensures that relationships between tables remain valid.
- **Supports multiple views** – Visitors and admins see only the data relevant to them.
- **Handles multiple transactions** – Several visitors can book at the same time without conflicts.

1.4.2 Advantages of DBMS

- **Data Security**: Provides access control and prevents unauthorized access.
- **Data Consistency**: Provides reliable data storage and retrieval.
- **Efficient Query Processing**: Allows queries that combine related data from many tables.
- **Scalability**: Can be extended as more sanctuaries, activities and visitors are added.

1.5 MySQL

1.5.1 Overview

MySQL is an open-source relational database management system (RDBMS) used for structured data storage. It is widely adopted because of its performance, security and scalability. WildBook uses the InnoDB storage engine, which supports transactions, row-level locking and foreign key constraints. These features are important for a reservation system, where many visitors may try to book the same slot at the same time.

1.5.2 SQL Commands Used in DBMS

Creating the Sanctuaries table

```
CREATE TABLE sanctuaries (
  sanctuary_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  location VARCHAR(150),
  description TEXT
);
```

Inserting a New Sanctuary Record

```
INSERT INTO sanctuaries (name, location, description)
VALUES ('Bandipur Tiger Reserve', 'Karnataka',
        'Forest reserve with safari activities');
```

Retrieving Activity Details

```
SELECT * FROM activities WHERE sanctuary_id = 1;
```

Updating Activity Information

```
UPDATE activities
SET price = 1800
WHERE activity_id = 2;
```

Deleting a Cancelled Booking Record

```
DELETE FROM bookings
WHERE booking_id = 5 AND status = 'CANCELLED';
```

1.5.3 SQL Statements

SQL provides essential commands to manage and manipulate relational databases. The key statements include:

1. **SELECT** – Retrieves information from a table.
```
SELECT * FROM activities WHERE sanctuary_id = 1;
```
2. **INSERT** – Adds new records to a table.
```
INSERT INTO guides (name, phone, experience)
VALUES ('Ravi Kumar', '9876543210', 8);
```
3. **DELETE** – Removes one or more existing records from a table.
```
DELETE FROM bookings
WHERE booking_id = 5 AND status = 'CANCELLED';
```
4. **UPDATE** – Modifies existing values in a table.
```
UPDATE bookings
SET status = 'CANCELLED'
WHERE booking_id = 101;
```

1.5.4 SQL Aggregate Functions

Aggregate functions perform calculations on a set of values and return a single result.

1. **COUNT** – Returns the number of records.
```
SELECT COUNT(*) FROM bookings;
```
2. **SUM** – Computes the total sum of values in a column.
```
SELECT SUM(amount) FROM payments
WHERE payment_status = 'PAID';
```
3. **MAX** – Returns the maximum value in a column.
```
SELECT MAX(price) FROM activities;
```
4. **MIN** – Returns the minimum value in a column.
```
SELECT MIN(capacity) FROM vehicles;
```
5. **AVG** – Calculates the average value in a column.
```
SELECT AVG(price) FROM activities;
```

1.5.5 SQL Constraints

Constraints ensure data integrity and enforce rules in a database.

1. **NOT NULL** – Ensures a column cannot have NULL values.
```
CREATE TABLE guides (
  guide_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL
);
```
2. **PRIMARY KEY** – Ensures a column has unique, non-null values.
```
CREATE TABLE vehicles (
  vehicle_id INT PRIMARY KEY,
  vehicle_type VARCHAR(50)
);
```
3. **UNIQUE** – Ensures all values in a column are distinct (no duplicates allowed).
```
CREATE TABLE users (
  email VARCHAR(100) UNIQUE,
  name VARCHAR(100)
);
```

4. **FOREIGN KEY** – Creates a link between two tables by referencing the primary key of another table.

```
CREATE TABLE activities (
  activity_id INT PRIMARY KEY,
  sanctuary_id INT,
  FOREIGN KEY (sanctuary_id) REFERENCES sanctuaries(sanctuary_id)
);
```

5. **CHECK** – Ensures that values in a column meet a specific condition.

```
CREATE TABLE vehicles (
  capacity INT CHECK (capacity > 0),
  vehicle_type VARCHAR(50)
);
```

6. **DEFAULT** – Assigns a default value to a column when no value is provided.

```
CREATE TABLE bookings (
  booking_id INT PRIMARY KEY,
  status VARCHAR(20) DEFAULT 'CONFIRMED'
);
```

7. **AUTO_INCREMENT** – Automatically generates a unique number when a new record is inserted (used mainly for primary keys).

```
CREATE TABLE guides (
  guide_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100)
);
```

8. **ENUM** – Restricts a column to a fixed list of allowed values.

```
CREATE TABLE payments (
  payment_id INT PRIMARY KEY,
  payment_status ENUM('PAID', 'FAILED', 'REFUNDED')
);
```

1.5.6 Triggers and Transactions

Some rules involve more than one table and cannot be written as a simple column constraint. WildBook uses **triggers** and **transactions** for these cases.

1. **TRIGGER** – A stored routine that MySQL runs automatically before or after an INSERT, UPDATE or DELETE. It is used to reject a slot whose capacity is larger than the capacity of its vehicle.

```
CREATE TRIGGER trg_slot_capacity BEFORE INSERT ON slots
FOR EACH ROW
BEGIN
  IF NEW.capacity > (SELECT capacity FROM vehicles
                     WHERE vehicle_id = NEW.vehicle_id) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Slot capacity exceeds vehicle capacity';
  END IF;
END;
```

2. **TRANSACTION** – A group of statements that either all succeed (COMMIT) or all fail (ROLLBACK). A booking is saved inside one transaction, and the slot row is locked with SELECT ... FOR UPDATE, so two visitors cannot take the last seats at the same moment.

```
START TRANSACTION;
SELECT capacity FROM slots WHERE slot_id = 12 FOR UPDATE;
-- count the seats already booked, then insert the booking
COMMIT;
```

CHAPTER 2
LITERATURE REVIEW

This chapter looks at the existing ways of managing safari and wildlife sanctuary reservations and establishes the software requirements and design concepts used in the WildBook project.

2.1 Survey of Existing Systems

Safari and activity reservations in many sanctuaries are still handled through paper registers, phone calls, counter bookings and basic spreadsheets. While these methods worked when the number of visitors was small, they have clear limitations in speed, accuracy and scalability, especially during holidays and peak tourist seasons when seats fill up quickly.

Manual systems often lead to overbooking, misplaced records, duplicate entries and long queues at the counter. It is also hard to keep track of which guide and which vehicle are free at a given time, and the identity details of visitors entering the forest are not always recorded properly. Online ticketing portals exist, but many sanctuaries still lack an integrated database that links activities, guides, vehicles, slots, bookings and payments in one place.

Disadvantages of Manual Systems:

- High chances of human error and manual miscalculation of seats and fees
- Overbooking of a safari slot beyond the capacity of the vehicle
- The same guide or vehicle being allotted to two slots at the same time
- Time-consuming data entry, searching and retrieval of records
- Poor security and access control for personal and ID details of visitors
- Repetition of data leading to inconsistencies
- Difficulty in tracking payments, refunds and cancellations

The proposed WildBook system aims to overcome these limitations by providing a centralized, automated and secure web-based solution for sanctuary management, slot scheduling, guide and vehicle allocation, booking with attendee details and payment recording.

2.2 Proposed System

In this project, WildBook is developed as a web-based application using React for the frontend, Node.js with the Express framework for the backend, and MySQL as the database management system.

React is used to build a responsive and user-friendly interface that allows visitors to move easily between sanctuaries, activities, slots and the booking form. Its component-based design keeps the pages reusable and the display up to date with the data. The Express framework, built on Node.js, serves as a lightweight backend that handles API requests, processes user data and connects with the database. MySQL, a reliable and scalable relational database system, stores sanctuaries, activities, guides, vehicles, slots, bookings, attendees and payments.

The proposed system manages the core reservation operations in one place: user registration, browsing of activities, slot scheduling, seat booking, attendee records and payment tracking. Every user, whether visitor or admin, is given role-based access to perform only the actions meant for that role. Business rules such as seat limits and one guide or vehicle per time slot are enforced by the database itself, so they hold no matter which screen or program changes the data.

2.3 Software Requirements

- **Operating System**: Windows / Linux / macOS
- **Frontend**: React (JavaScript, HTML, CSS)
- **Backend**: Node.js with the Express framework
- **Database Management System**: MySQL (InnoDB storage engine)

- **Web Server**: Express.js server (Node-based)
- **Development Tools**: Visual Studio Code, MySQL Workbench, Git, npm
- **Browser**: Any modern web browser such as Chrome, Edge or Firefox

2.4 Frontend Technologies

The frontend is the interface through which visitors and admins interact with WildBook. It is responsible for a smooth, responsive and intuitive experience across different devices and browsers. The frontend is built using the following web technologies.

1. HTML and CSS

Hypertext Markup Language (HTML) structures the content displayed on the web pages, such as headings, forms, tables, buttons and input fields. Cascading Style Sheets (CSS) style and organize these elements, controlling fonts, colours, spacing and the responsive layout so that the application works well on both desktop and mobile screens.

2. JavaScript

JavaScript adds interactivity to the web application. In this system it is used for form validation, input checking (for example, attendee age and ID fields) and communication with the backend APIs.

3. React

React is a JavaScript library for building user interfaces from reusable components. In WildBook it is used to build the pages for browsing sanctuaries, viewing slots, booking with attendee details and the admin dashboard. React updates only the parts of the page whose data has changed, which gives a fast and smooth experience, and it renders content according to the role of the logged-in user.

2.5 Backend Technologies

The backend of WildBook handles the core logic of the application: processing client requests, managing authentication, interacting with the database and returning responses to the frontend. It makes sure that operations such as booking, slot creation and payment recording are carried out securely and correctly.

1. Node.js with Express

Node.js is a JavaScript runtime that lets the backend be written in the same language as the frontend, and Express is a minimal and flexible web framework built on top of it. Together they are used to develop the web server and APIs of WildBook, with simple routing, request handling and middleware.

Node.js with Express provides:

- A lightweight and efficient framework for backend development.
- Built-in routing for handling client requests and API endpoints.
- Easy integration with MySQL for database operations through a MySQL driver.
- Support for JSON-based communication with the frontend.
- Secure handling of user authentication and sessions.

2. MySQL

MySQL is an open-source relational database management system used to store the structured data of WildBook. It organizes data in tables and supports complex queries for efficient retrieval and processing. MySQL is reliable and scalable, and with the InnoDB engine it supports transactions, which are crucial for booking seats safely.

MySQL offers:

- Structured storage of sanctuary, slot, booking, attendee and payment data.
- Foreign key constraints to ensure referential integrity.

- CHECK constraints, ENUM columns and triggers to enforce domain and business rules.
- Row-level locking and transactions for safe concurrent bookings.
- Secure login and access control features.

2.6 Entity-Relationship (ER) Model

The Entity-Relationship (ER) model is a high-level conceptual data model that provides a systematic approach to database design by defining the data elements, their attributes and the relationships between them. It serves as a blueprint for constructing relational databases and is especially useful for capturing the requirements of a reservation system such as WildBook.

Purpose of the ER Model

The main goal of the ER model is to:

- Represent the data structure in a way that both technical and non-technical stakeholders can understand.
- Identify the entities that need to be represented in the database.
- Define the relationships between these entities.
- Specify the attributes of each entity and relationship.
- Establish integrity constraints such as keys and cardinalities.

Core Components of the ER Model

The ER model consists of three fundamental components: entities, attributes and relationships.

1. Entities

Entities are objects or concepts that are distinguishable from other objects. Each entity represents a set of real-world objects that share the same properties. In WildBook, typical entities include User, Sanctuary, Activity, Guide, Vehicle, Slot, Booking and Payment.

- **Strong Entity**: Has a primary key that uniquely identifies each instance (e.g., Sanctuary, Guide).
- **Weak Entity**: Cannot be uniquely identified without a related strong entity (e.g., an attendee that is identified only through its booking).

Entities are represented as rectangles in an ER diagram.

2. Attributes

Attributes are the properties or characteristics of an entity or relationship. For example, a Guide entity may have attributes such as guide_id, name, phone and experience.

Types of attributes include:

- **Simple (Atomic)**: Cannot be divided further (e.g., guide_id).
- **Composite**: Can be broken down into smaller sub-parts (e.g., Location → City and State).
- **Derived**: Can be calculated from other attributes (e.g., seats left in a slot, calculated from capacity and confirmed bookings).
- **Multivalued**: May have more than one value for an entity instance (e.g., several attendees in one booking).

Attributes are typically represented as ellipses in an ER diagram, connected to their entity or relationship.

3. Relationships

Relationships describe how two or more entities are associated with each other. In WildBook, examples of relationships include:

- A User makes one or more Bookings.
- A Sanctuary offers one or more Activities.
- A Slot uses one Guide and one Vehicle for an Activity.
- A Booking is paid by one Payment.

Relationships are depicted as diamonds in an ER diagram, connected to the participating entities. Each relationship can have:

- **Cardinality constraints**: Define the number of instances of one entity that can or must be associated with instances of another entity.
  - One-to-One (1:1): Each instance on one side is related to at most one instance on the other side.
  - One-to-Many (1:N): One instance of an entity is associated with many instances of another.
  - Many-to-Many (M:N): Many instances of one entity are related to many instances of another.
- **Participation constraints**: Specify whether all instances of an entity must take part in the relationship.
  - Total participation: Every instance of the entity must be involved in the relationship.
  - Partial participation: Some instances may not take part.

Keys and Integrity Constraints

- **Primary Key (PK)**: An attribute or set of attributes that uniquely identifies an entity instance.
- **Foreign Key (FK)**: An attribute that creates a link between entities by referring to the primary key of another entity.
- **Referential Integrity**: Ensures that relationships between tables remain consistent (a foreign key must match an existing primary key or be null).

Advanced Concepts in the ER Model

- **Generalization**: Extracting shared characteristics from two or more entities to create a generalized entity.
- **Specialization**: The opposite of generalization, where a higher-level entity is divided into sub-entities based on distinguishing characteristics.
- **Aggregation**: A higher-level abstraction that treats a relationship set as an entity set so that it can take part in other relationships.
- **Weak Entity and Identifying Relationships**: Used when an entity cannot be uniquely identified by its own attributes and relies on a related entity and a partial key.

Benefits of Using an ER Model in WildBook

- Facilitates a clear understanding of the system's data structure.
- Ensures logical consistency and accuracy of data.
- Helps in identifying redundant data and removing anomalies through normalization.
- Acts as a reference model during database implementation.
- Supports maintenance and scalability by providing a well-organized schema.

2.7 Relational Schema Design

The relational schema is a representation of the database structure in terms of relations (tables), where data is organized in rows and columns. It is derived from the ER model and forms the logical structure that is eventually implemented in a relational database system.

Each entity and relationship in the ER model is translated into one or more relations, and constraints such as primary keys, foreign keys and cardinality are enforced through the schema definitions. The goal is to create a normalized, non-redundant and integrity-preserving relational schema that accurately models the domain.

7-Step Approach to Convert ER Model to Relational Schema

Converting an ER model to a relational schema is a systematic process. The following seven rules outline a structured approach, and each rule is shown together with how it applies to WildBook.

Rule 1: Mapping Regular (Strong) Entity Sets

- Each strong entity in the ER model becomes a separate relation (table).
- All simple and composite attributes are included as columns; composite attributes are flattened into individual attributes.
- A primary key is chosen to uniquely identify each tuple.

Example:

```
Sanctuary(sanctuary_id PRIMARY KEY, name, location, description)
```

In WildBook: users, admin_profiles, sanctuaries, activities, guides, vehicles, slots, bookings, and payments form the 9 normalized tables of the relational schema.

Rule 2: Mapping Weak Entity Sets

- Create a relation for the weak entity with its partial key.
- Include the primary key of the owner entity as a foreign key.
- The foreign key and the partial key together form the composite primary key.

Example:

```
Attendee(booking_id, attendee_no, name, age,
         PRIMARY KEY (booking_id, attendee_no),
         FOREIGN KEY (booking_id) REFERENCES Booking)
```

In WildBook: weak entity structures (such as individual attendee lists) are consolidated into atomic attributes (number_of_people on bookings), eliminating redundant tables while maintaining 1NF/2NF/3NF compliance.

Rule 3: Mapping Binary 1:1 Relationships

- The foreign key is placed in one of the two tables, preferably the one with total participation.
- A UNIQUE constraint is added on the foreign key.

Example:

```
Payment(payment_id PRIMARY KEY, booking_id UNIQUE,
        FOREIGN KEY (booking_id) REFERENCES Booking)
```

In WildBook: a booking and its payment form a 1:1 relationship, so booking_id is placed in payments and made UNIQUE. Similarly, admin_profiles forms a 1:1 extension of users, with user_id as a UNIQUE foreign key.

Rule 4: Mapping Binary 1:N Relationships

- The foreign key is placed in the relation on the 'many' side.
- The foreign key references the primary key of the 'one' side, which ensures referential integrity.

Example:

```
Activity(activity_id PRIMARY KEY, sanctuary_id,
         FOREIGN KEY (sanctuary_id) REFERENCES Sanctuary)
```

In WildBook: sanctuary → activities and vehicles; activity, guide and vehicle → slots; user → bookings; slot → bookings.

Rule 5: Mapping Binary M:N Relationships

- A separate relation (junction table) is created for the relationship.
- It includes foreign keys referencing both participating entities, plus any attributes of the relationship.

Example:

```
Booking(booking_id PRIMARY KEY, user_id, slot_id,
        number_of_people, status, booked_at,
        FOREIGN KEY (user_id) REFERENCES User,
        FOREIGN KEY (slot_id) REFERENCES Slot)
```

In WildBook: many users can book many slots, so bookings is the junction table, and number_of_people, total_amount, status, hold_expires_at, cancelled_at, cancellation_fee, and refund_amount are relationship attributes.

Rule 6: Mapping Multivalued Attributes

- A multivalued attribute is moved into a separate relation.
- The new relation holds the attribute together with the primary key of its parent as a foreign key.

Example:

```
Booking_Attendee(attendee_id PRIMARY KEY, booking_id, name, age,
                 FOREIGN KEY (booking_id) REFERENCES Booking)
```

In WildBook: multivalued document uploads for operator onboarding are normalized as atomic columns (license_doc, identity_doc, auth_letter_doc) in admin_profiles, while headcount is an atomic scalar integer in bookings, preventing multivalued repeating groups without unnecessary tables.

Rule 7: Mapping Ternary (and Higher) Relationships

- For a relationship among three or more entities, a new relation is created.
- The primary keys of all participating entities become foreign keys, and together form the key of the new relation.

In WildBook: this rule is not needed. A slot is treated as an entity with three binary links (to activity, guide and vehicle), so no ternary relationship remains.

This 7-step method ensures a systematic and consistent transformation of the ER model into a fully functional relational schema. It preserves data integrity through keys and constraints, logical consistency by maintaining relationships, and efficiency through normalization and redundancy elimination. The resulting schema becomes the foundation for the physical implementation in MySQL.

2.8 Normalization

Normalization is a database design technique that reduces data redundancy and eliminates undesirable characteristics such as insertion, update and deletion anomalies. Normalization rules divide larger tables into smaller tables and link them using relationships.

Normalization of data can be looked upon as a process of analyzing the given relation schemas based on their functional dependencies and primary keys to achieve the desirable properties of:

- Minimizing redundancy.
- Minimizing the insertion, deletion and update anomalies.

Functional Dependency

A functional dependency, denoted by X → Y, between two sets of attributes X and Y that are subsets of a relation R specifies a constraint on the possible tuples of R: for any two tuples t1 and t2 with t1[X] = t2[X], it must also hold that t1[Y] = t2[Y]. In other words, the values of X uniquely determine the values of Y.

The normal forms used in relational database design are:

- 1NF (First Normal Form)
- 2NF (Second Normal Form)
- 3NF (Third Normal Form)
- BCNF (Boyce-Codd Normal Form)
- 4NF (Fourth Normal Form)
- 5NF (Fifth Normal Form)

a) First Normal Form (1NF)

A relation is in 1NF if the domain of every attribute includes only atomic values and the value of every attribute in a tuple is a single value from that domain. 1NF disallows sets of values, lists, repeating groups or combinations of these inside a single attribute value.

b) Second Normal Form (2NF)

This normal form is based on full functional dependency. A dependency X → Y is fully functional if the removal of any attribute A from X means that the dependency no longer holds. A relation R is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the primary key, that is, no non-key attribute depends on only a part of a composite key.

c) Third Normal Form (3NF)

Third Normal Form is based on the concept of transitive dependency. A dependency X → Y in R is transitive if there is a set of attributes Z that is neither a candidate key nor a subset of any key of R, and both X → Z and Z → Y hold. A relation R is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key.

d) Boyce-Codd Normal Form (BCNF)

A relation R is in BCNF if, for every one of its functional dependencies X → Y, X is a super key. BCNF is a stricter version of 3NF: every determinant must be a candidate key.

e) Fourth Normal Form (4NF)

A relation is in 4NF if it is in BCNF and has no multi-valued dependencies. A multi-valued dependency occurs when one attribute determines a set of values of another attribute independently of the remaining attributes.

f) Fifth Normal Form (5NF)

A relation is in 5NF if it is in 4NF and has no join dependencies that are not implied by the candidate keys. This form removes redundancy caused by complicated relationships among the data.

Normalization of the WildBook Database

The WildBook schema was designed in a fixed order: the ER diagram was converted into draft tables using the 7 conversion rules, the draft tables were audited with 1NF, 2NF, 3NF and BCNF, and only then was the final relational schema drawn. The diagram is drawn last because normalization can still split tables and add foreign keys. The result of the audit is summarized below.

Rule	What it demands	Result on WildBook
1NF	Every column holds one atomic value; no repeating groups.	Satisfied. In our streamlined schema, the attendee headcount is stored as an atomic integer column number_of_people, avoiding repeating groups while keeping all 9 tables in strict 1NF.
2NF	No non-key column depends on part of a composite key.	Satisfied. Every table has a single-column primary key, so nothing needs to be split.
3NF	No non-key column depends on another non-key column.	Satisfied. Guide, vehicle, activity, sanctuary and user details each live in their own table.
BCNF	Every determinant is a candidate key.	Satisfied after the real candidate keys are declared: UNIQUE on user email, on (guide, date, time) and (vehicle, date, time) in slots, and on the booking in payments.

Rule	What it demands	Result on WildBook
Integrity	Domains and cross-table rules must still be enforced.	CHECK constraints, ENUM status columns, a role column, ON DELETE rules and triggers on slots are added.

In short, the 7 conversion rules already did most of the splitting. The audit found one real normal-form violation (1NF), and the remaining work was declaring candidate keys and constraints.

Example of the 1NF Violation

If attendees were stored as comma-separated text strings within bookings, for example attendee_names = 'Asha, Ravi, Meena', 1NF would be violated due to non-atomic values. In WildBook, this is resolved by storing an atomic integer number_of_people, guaranteeing strict 1NF compliance. This design has the following problems:

- One attendee cannot be searched or indexed, and rules such as "age must be greater than 0" cannot be enforced per person.
- Changing one person's name means editing a text list (update anomaly), and the schema does not limit the number of attendees.
- The fix is Rule 6: every multivalued attribute becomes a child table with the parent's key as a foreign key.

What a 3NF Violation Would Have Looked Like

- If slots stored the guide's name and phone, then slot_id → guide_id → guide_name would be a transitive dependency. Renaming a guide would need many row updates, so this data stays in guides.
- If bookings stored the user's email, one email change would touch every booking of that user, so the email stays in users.
- If activities stored the sanctuary name, the name would repeat for every activity, so it stays in sanctuaries.

Anomalies Prevented by the Final Design

- **Insertion**: a guide, vehicle or sanctuary can be added before it has any slot or activity.
- **Update**: a sanctuary name, activity price or user email is stored in exactly one place.
- **Deletion**: cancelling a booking changes its status; the booking and payment rows are kept, so history is not lost.

Controlled Redundancy

Some redundancy is kept on purpose. These items are not normalization failures, because each one has a reason and a way of being kept correct.

Item	Why it is kept	How it is kept correct
payments.amount	A price snapshot (price × people at booking time). Prices change later, but a refund must match what was paid.	Written once in the booking transaction.
bookings.number_of_people	Stored as an atomic headcount integer to check slot capacity and calculate booking totals.	Checked against remaining slot capacity inside sp_reserve_booking using SELECT ... FOR UPDATE row-level locking.
Seats booked per slot	Not stored. A column such as booked_count would be derived data that can go stale.	Computed as the SUM of number_of_people of CONFIRMED and active

Item	Why it is kept	How it is kept correct
		PENDING bookings under SELECT ... FOR UPDATE.

2.9 API Development

To enable communication between the frontend and backend, WildBook uses a set of RESTful APIs developed with Node.js and Express. These APIs act as the bridge between the React user interface and the MySQL database, allowing real-time data exchange for the reservation operations. Data is exchanged in JSON format and the standard HTTP methods (GET, POST, PUT and DELETE) are used.

The backend handles CRUD (Create, Read, Update, Delete) operations for the following groups of data:

- User registration, login and role-based access for visitors and admins.
- Sanctuaries, activities, guides and vehicles.
- Slots, including the allocation of a guide and a vehicle.
- Bookings with attendee details, cancellation and the related payment records.

The booking API performs its work inside a single database transaction: it locks the slot, checks the seats that are left, inserts the booking and its attendees, and records the payment. If any step fails, the whole transaction is rolled back, so the database never holds a half-finished booking.

2.10 Advantages of the Proposed System

The proposed WildBook system provides several advantages over traditional manual reservation processes. By combining modern web technologies with a centralized, well-designed database, the system improves efficiency, reliability and user experience.

- **Improved Accuracy and Efficiency**
  All sanctuary, slot and booking information is stored in a structured MySQL database, which minimizes human error and gives quick and accurate retrieval of data.
- **No Overbooking or Double Allocation**
  Capacity checks, triggers and unique keys make sure that a slot never carries more people than its vehicle can hold and that a guide or vehicle is never assigned twice at the same time.
- **Enhanced Data Security**
  Secure login, password hashing and role-based access protect visitor information, and ID numbers are shown only in part.
- **Real-Time Data Access**
  Visitors can see open slots, book seats and check booking status at any time, which improves convenience.
- **Normalized and Consistent Data**
  The schema is normalized up to 3NF/BCNF, so every fact is stored in one place and update, insertion and deletion anomalies are avoided.
- **Complete Booking History**
  Cancelling a booking only changes its status, so booking and payment records are kept for reports and audits.
- **Scalable and Modular Design**
  The system is modular, so new sanctuaries, activities and features such as reviews or notifications can be added easily in the future.

CHAPTER 3
PROPOSED WORK

This chapter presents the design and structure of WildBook, explaining how the data is organized and how the entities interact. It outlines the relationships modelled in the system and shows how users, sanctuaries, activities, guides, vehicles, slots, bookings, attendees and payments are connected for smooth safari reservation.

3.1 Entity Relationship (ER) Model

The ER diagram shows how the key entities and their attributes are connected to ensure smooth information flow and reduced redundancy. The design improves data consistency, maintenance and retrieval efficiency.

Entities

The proposed system includes the following entities, each with specific attributes that store the data needed for reservation operations:

- **User:** A visitor, sanctuary administrator, or site owner registered in the system with role-based access control.
- **Sanctuary**: A wildlife sanctuary that offers activities and owns vehicles.
- **Activity**: A safari or other activity offered in a sanctuary, with its price.
- **Guide**: A guide who leads the slots.
- **Vehicle**: A safari vehicle with a seating capacity.
- **Slot**: A scheduled run of an activity on a date and time with one guide and one vehicle.
- **Booking:** A reservation of safari slot seats made by a user, containing atomic attendee headcount, 10-minute hold timestamp, and cancellation financial fields.
- **Admin_Profile:** An operator verification profile linked 1:1 with User, storing agency name, designation, and verification documents (license, photo ID, authorization letter).
- **Payment**: The payment made for a booking.

All entities are stored in a MySQL relational database linked through primary and foreign keys, enabling seamless data access, real-time updates and strong data integrity.

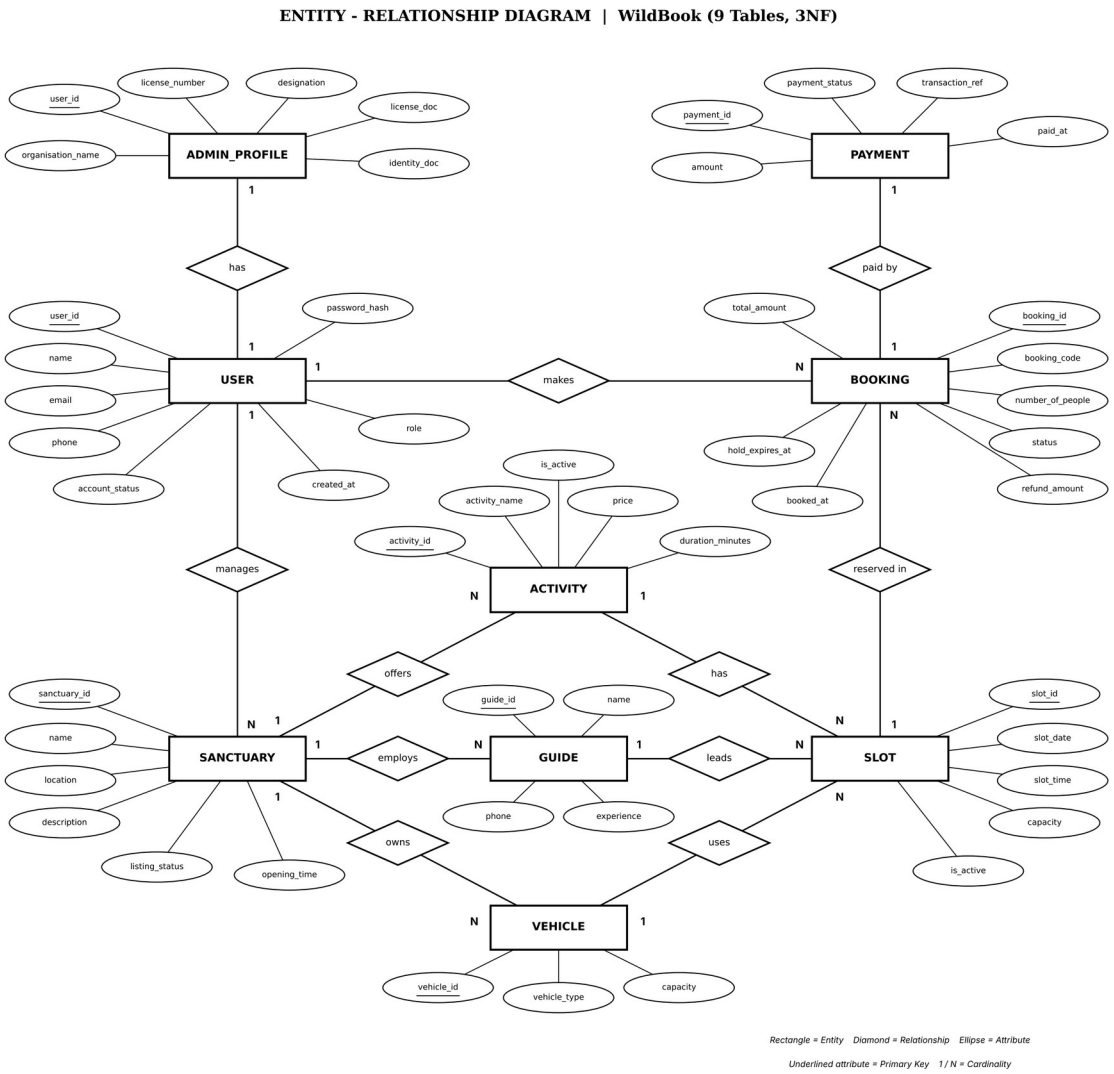


Figure 1: ER Diagram of WildBook

3.2 Relational Model

The relational model defines the structure of the database in terms of tables, their attributes and the relationships between them. WildBook is built using relational tables to store, retrieve and manage the data of sanctuaries, activities, guides, vehicles, slots, bookings, attendees and payments. The tables are normalized and interconnected using primary keys and foreign keys, which ensures data integrity, consistency and minimal redundancy.

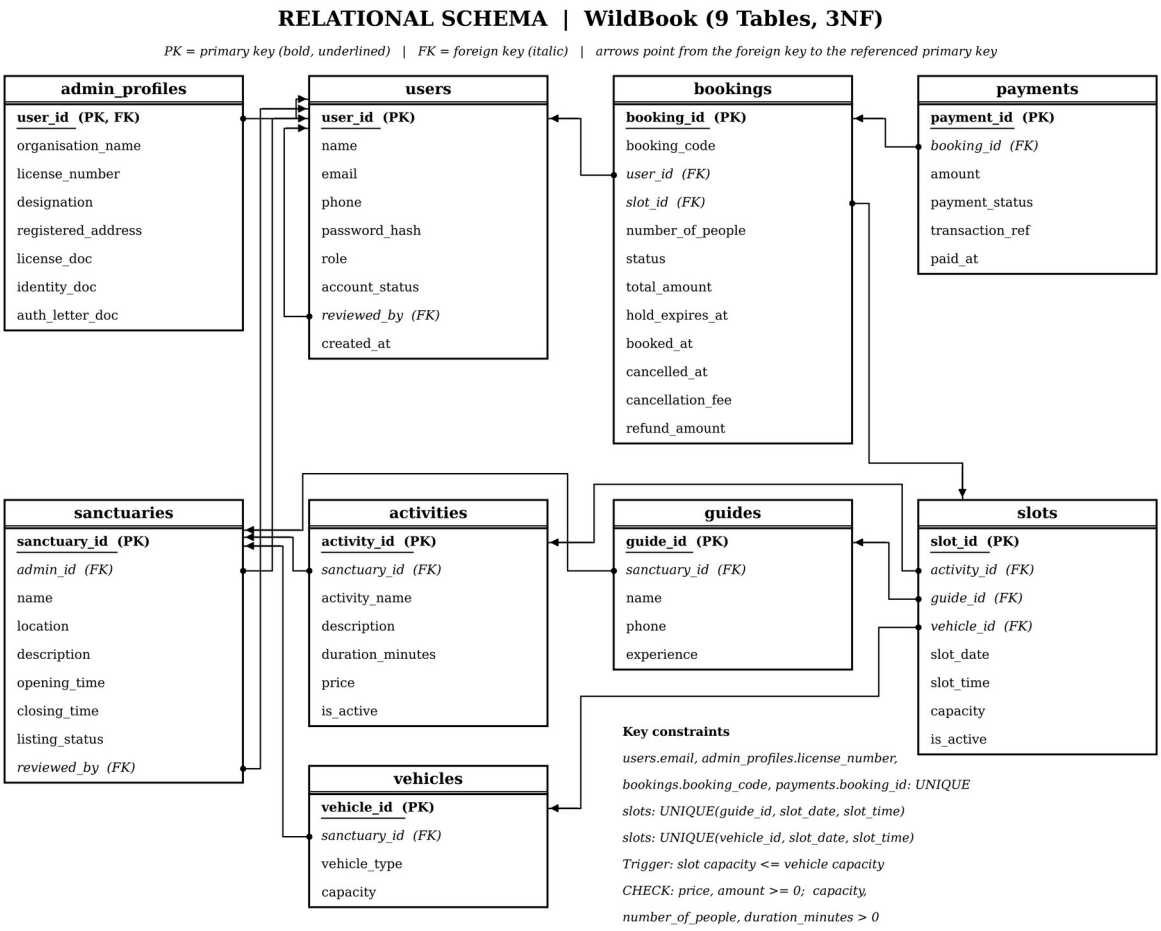


Figure 2: Relational Model of WildBook

Relationships and Cardinalities

1. A User can make multiple Bookings (User → Booking) (1:N)

- Each user can make several bookings, but every booking belongs to one specific user.

2. A Slot can have multiple Bookings (Slot → Booking) (1:N)

- Many bookings can be made for one slot, but each booking is for exactly one slot. Together with Relationship 1, this makes Booking the junction table of the M:N relationship between User and Slot.

3. A User has at most one Admin Profile (User → Admin_Profile) (1:1)

- An administrator user has exactly one administrative profile record detailing agency credentials and verification documents.

4. A Booking has one Payment (Booking → Payment) (1:1)

- Every booking has one payment record, and each payment is linked to a specific booking.

5. A Sanctuary offers multiple Activities and owns multiple Vehicles (1:N)

- Each activity and each vehicle belongs to one sanctuary, while a sanctuary can have many of both.

6. An Activity, a Guide and a Vehicle each appear in multiple Slots (1:N)

- Each slot has exactly one activity, one guide and one vehicle, while the same guide or vehicle can be used in many slots at different times.

CHAPTER 4
RESULT

4.1 Outcomes

WildBook was designed and implemented as a web application with a MySQL database to manage the complete safari reservation cycle: sanctuaries and activities, guides and vehicles, slot scheduling, bookings with 10-minute seat holds, cancellation management, and payments across 9 normalized tables. By automating these tasks, the system reduces manual effort and removes the chances of duplicate data or human error during booking and payment.

The database enforces the main reservation rules by itself. A guide or a vehicle cannot be allotted to two slots at the same date and time, the capacity of a slot cannot exceed that of its vehicle, a booking cannot take more seats than are left, each booking has at most one payment, and every attendee has an age and an ID recorded. Status columns accept only the defined values, and prices, ages and capacities cannot be negative.

Visitors can browse sanctuaries and activities, view slots, book seats and pay through a simple interface, while administrators can manage the master data, create slots and monitor bookings and payments. Cancelling a booking changes its status instead of deleting it, so that booking and payment history is preserved for reports.

WildBook therefore provides a reliable, scalable and secure platform for visitors and administrators, and shows how a well-normalized relational database can support a real-world reservation system.

4.2 Screenshots

The following is a series of screenshots of the developed application.

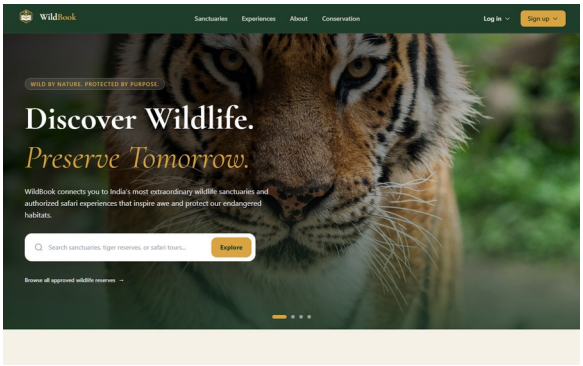


Figure 3: Welcome Page

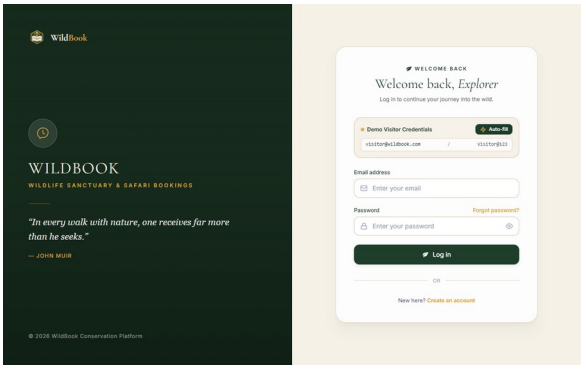


Figure 4: User Registration and Login Page

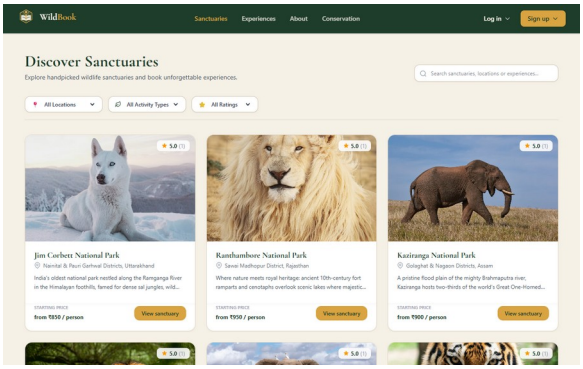


Figure 5: Sanctuaries and Activities Page

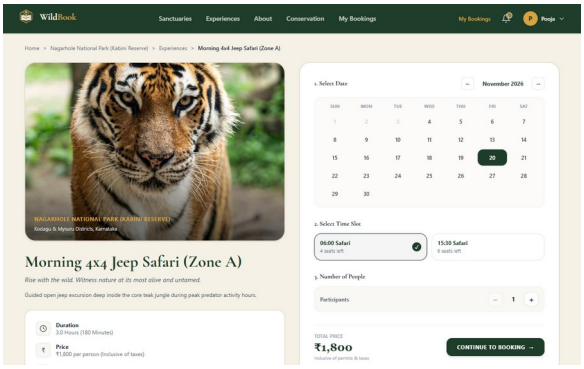


Figure 6: Slot Booking Page with Live Calendar and Seat Selection

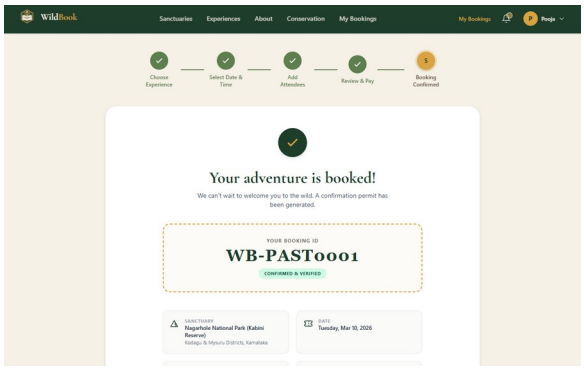


Figure 7: Payment and Booking Confirmation

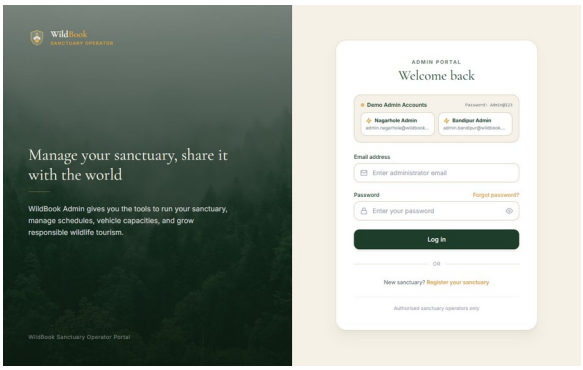


Figure 8: Admin Dashboard

Figure 3: Welcome Page – Introduces users to the WildBook portal and provides the entry point to login and registration.

Figure 4: User Registration and Login Page – Allows new visitors to register and existing users and admins to log in securely.

Figure 5: Sanctuaries and Activities Page – Displays the sanctuaries, the activities they offer, their prices and the open slots.

Figure 6: Slot Booking Page – Enables a visitor to select a date, inspect available morning/afternoon slots, select headcount, and reserve with an automated 10-minute hold.

Figure 7: Payment and Booking Confirmation – Records the payment for the booking and shows the confirmed booking details.

Figure 8: Admin Dashboard – Allows the administrator to manage sanctuaries, activities, guides, vehicles and slots and to monitor bookings and payments.

CONCLUSION

WildBook, the Wildlife Sanctuary & Safari Reservation Management System, was developed to streamline and automate the key operations of safari reservation. The project integrates a MySQL relational database with a user-friendly frontend built using React, and a backend powered by Node.js and Express. This combination provides a smooth link between visitors and administrators and an organized platform for managing wildlife activities.

The system handles sanctuary and activity management, guide and vehicle allocation, slot scheduling, seat booking with attendee records and payment tracking. By following the ER-to-relational conversion rules and normalizing the schema up to 3NF/BCNF, WildBook minimizes data redundancy, ensures consistency and allows quick and reliable retrieval of data. Constraints, triggers and transactions protect the data against overbooking, double allocation and invalid values.

Through its clear design and interactive interface, the system lets both administrators and visitors manage reservations easily. WildBook not only simplifies safari booking and sanctuary management but also gives visitors transparency and convenience throughout the process.

Future Enhancements

While the current version handles the core reservation functions, future improvements may include:

- Integration of a real-time online payment gateway.
- Automated email/SMS notifications for booking confirmations and slot changes.
- Development of a mobile application for better accessibility.
- Visitor reviews and ratings for guides and activities.
- Analytics dashboards and reports for administrators to track bookings, revenue and popular activities.

In conclusion, WildBook provides a centralized, secure and scalable platform that improves safari reservation, protects the integrity of booking data and ensures smooth coordination among everyone involved. It stands as a reliable and efficient solution for modern wildlife sanctuary management.

BIBLIOGRAPHY

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill, 2019.

[3] R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed. New York: McGraw-Hill, 2003.

[4] E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970.

[5] P. P. Chen, “The entity-relationship model – toward a unified view of data,” ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9–36, 1976.

[6] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Mateo, CA: Morgan Kaufmann, 1993.

[7] Oracle Corporation, MySQL 8.0 Reference Manual (InnoDB storage engine, constraints, triggers, stored procedures and locking reads). [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[8] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000.

[9] OpenJS Foundation, Node.js Documentation. [Online]. Available: https://nodejs.org/docs/latest/api/

[10] OpenJS Foundation, Express 5.x API Reference. [Online]. Available: https://expressjs.com/en/5x/api.html

[11] mysql2 contributors, MySQL2 – MySQL client for Node.js (connection pools and prepared statements). [Online]. Available: https://sidorares.github.io/node-mysql2/docs

[12] Meta Platforms, Inc., React Documentation. [Online]. Available: https://react.dev

[13] Vite and Tailwind Labs, Vite Guide and Tailwind CSS Documentation. [Online]. Available: https://vite.dev/guide/ and https://tailwindcss.com/docs

[14] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[15] N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annual Technical Conference (FREENIX Track), Monterey, CA, USA, 1999, pp. 81–91.